

Week 8

1. Contenidor Nginx

El contenidor de Nginx es basa en la imatge oficial `nginx:latest`. Això permet utilitzar un servidor web ja preparat i optimitzat sense haver de configurar-lo manualment des de zero.

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
```

Explicació

- **FROM nginx:latest**
Aquesta línia defineix la imatge base. La imatge oficial de Nginx és estable, lleugera i àmpliament utilitzada, cosa que la fa una opció fiable.
- **COPY index.html /usr/share/nginx/html/index.html**
Aquesta instrucció copia un fitxer HTML personalitzat dins del contenidor, substituint la pàgina per defecte de Nginx.

Decisions de disseny

- S'utilitza la imatge oficial per evitar configuracions innecessàries i reduir la complexitat.
 - Només es copia el fitxer necessari (`index.html`), mantenint la imatge simple i eficient.
 - No s'afegeix configuració extra perquè la configuració per defecte de Nginx ja és suficient per servir contingut estàtic.
-

2. Contenidor d'aplicació Python

Aquest contenidor executa un servidor HTTP senzill escrit en Python, que respon a peticions amb un missatge simple.

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY app.py .
```

```
EXPOSE 8000
```

```
CMD ["python", "app.py"]
```

Explicació

- **FROM python:3.11-slim**
S'utilitza una imatge de Python lleugera per reduir la mida del contenidor.
- **WORKDIR /app**
Defineix el directori de treball dins del contenidor.
- **COPY app.py .**
Copia el codi de l'aplicació dins del contenidor.
- **EXPOSE 8000**
Indica que el contenidor utilitza el port 8000.
- **CMD ["python", "app.py"]**
Defineix la comanda que s'executa quan s'inicia el contenidor.

Decisions de disseny

- S'utilitza la versió `slim` de Python per optimitzar la mida de la imatge.
 - L'aplicació és minimalista per centrar-se en l'aprenentatge de la containerització.
 - Es fa servir el port 8000, habitual en entorns de desenvolupament.
 - El servidor escolta a `0.0.0.0` per permetre connexions des de fora del contenidor.
-

3. Optimització dels Dockerfiles

Abans

Amb la comanda `docker images` podem veure informació com la mida de cada contenidor (imatge):

- `nginx-gsx` = 161MB
- `simple-app-gsx` = 124MB

Canvis realitzats en l'aplicació

1. Canvi d'imatge base

S'ha canviat la imatge base per: `FROM python:3.11-alpine`

Aquesta imatge és més lleugera perquè Alpine és una distribució mínima que inclou només el necessari. Això permet reduir la mida final de la imatge, millorar l'eficiència i disminuir la superfície d'atac

2. Millora de seguretat

Per defecte, els contenidors s'executen com a root, cosa que suposa un risc de seguretat. Ara seguirem el principi de **mínim privilegi**, reduint l'impacte en cas de vulnerabilitat.

Per evitar-ho, s'ha creat un usuari no privilegiat:

```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup \  
&& chown appuser:appgroup /app/app.py
```

A continuació, es configura el contenidor perquè s'executi amb aquest usuari:

```
USER appuser
```

3. Resultat final APP

Tornem a recompilar el contenidor amb la comanda: `docker build -t simple-app-gsx .`
I amb la comanda `docker images simple-app-gsx`, podem veure que la mida a sigut reduïda 54.3MB, reducció de més del **50% de la mida**

Canvis realitzats en NGINX

1. Canvi d'imatge base

S'ha substituït `nginx:latest` per `nginx:alpine` per reduir la mida, millorar el rendiment i evitar l'ús de `latest`, ja que és bona pràctica.

2. Resultat final NGINX

Tornem a recompilar el contenidor amb la comanda: `docker build -t nginx-gsx .`
I amb la comanda `docker images nginx-gsx`, podem veure que la mida a sigut reduïda 62.2MB, reducció de més del **50% de la mida**

Decisions de disseny

No s'ha aplicat multistage build en aquest cas perquè l'aplicació és molt simple i no requereix cap fase de compilació ni separació entre build i runtime.

Aquest tipus d'optimització és més útil en aplicacions amb dependències complexes o processos de compilació, cosa que no es dona en aquest projecte.

4. Dependències de cada aplicació

Contenidor Nginx: No té dependències externes. La imatge `nginx:alpine` ja inclou tot el necessari per servir contingut estàtic. L'únic fitxer extern que necessita és `index.html`, que es copia dins el contenidor durant la construcció.

Contenidor Python: Tampoc té dependències externes de Python (el `requirements.txt` està buit). La imatge `python:3.11-alpine` ja inclou l'interpret de Python necessari per executar `app.py`. L'aplicació utilitza únicament la llibreria estàndard de Python (`http.server`), que ve inclosa per defecte.

5. Passos per construir i executar localment

Nginx:

```
docker build -t nginx-gsx .
```

```
docker run -p 80:80 nginx-gsx
```

Desde una altra terminal: `curl localhost` (s'ha de veure la pàgina)

Aplicació Python:

```
docker build -t simple-app-gsx .
```

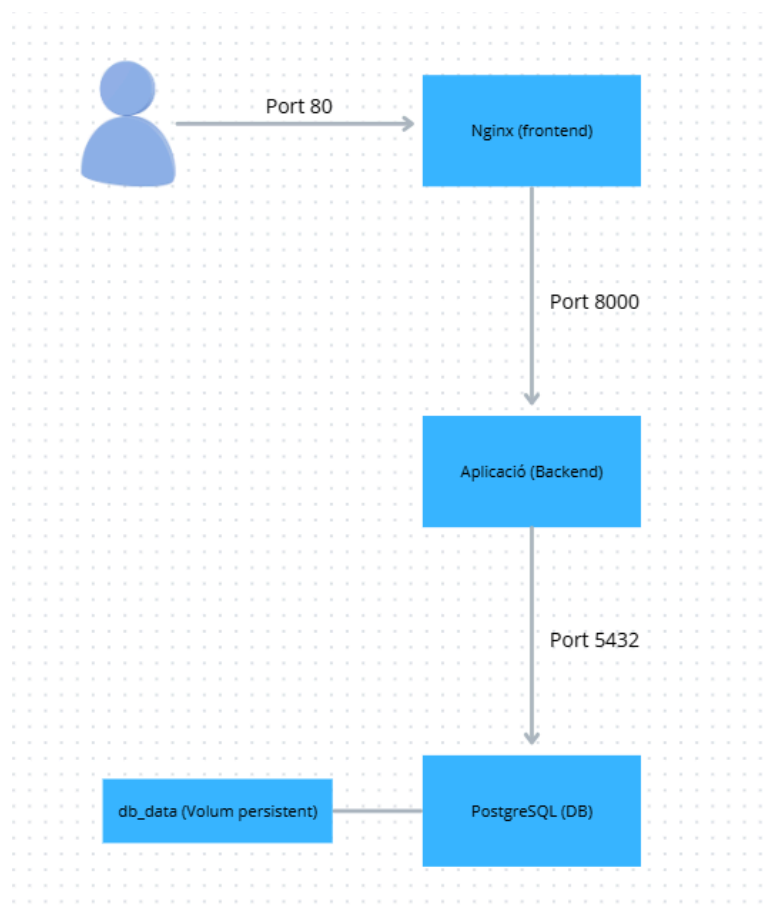
```
docker run -p 8000:8000 simple-app-gsx
```

Desde una altra terminal: `curl localhost:8000`

Week 9

1. Architecture diagram

L'arquitectura del sistema està formada per tres serveis principals: **nginx**, **backend** i **db**. L'usuari accedeix a Nginx, que actua com a frontend, també redirigeix les peticions al backend, que és una aplicació Python encarregada de consultar la base de dades PostgreSQL. Les dades de la base de dades es guarden en un volum persistent anomenat **db_data**. A continuació l'esquema:



2. Services

2.1 Nginx

El servei **nginx** actua com a punt d'entrada del sistema. Serveix la pàgina web i redirigeix les peticions de l'usuari cap al backend, que processa la informació. Això permet separar les diferents parts de l'aplicació i millorar l'organització del sistema.

2.2 Backend

El servei **backend** és una **aplicació Python** que processa les peticions i accedeix a la base de dades. En aquest projecte, l'aplicació consulta la taula **students** de PostgreSQL i retorna els noms emmagatzemats. És necessària perquè conté la lògica del sistema i fa d'intermediari entre Nginx i la base de dades.

2.3 Database

El servei **db** correspon a una base de dades PostgreSQL. La seva funció és emmagatzemar les dades de manera persistent. En aquest cas, s'utilitza per guardar els noms dels integrants del grup dins la taula **students**. És necessari perquè permet separar les dades de l'aplicació i mantenir-les encara que els contenidors es reiniciïn.

3. Data persistence

Per garantir la persistència de les dades, s'ha configurat un volum anomenat **db_data**. Aquest volum està associat al directori **/var/lib/postgresql/data** del contenidor de PostgreSQL, que és on la base de dades guarda la seva informació.

Les dades que es mantenen són els registres de la taula **students**, en aquest cas els noms introduïts a la base de dades (**Nicolas** i **Marçal**). Aquestes dades persisteixen perquè no es guarden dins del contenidor, sinó dins del volum **db_data**, que Docker conserva encara que els contenidors s'aturin o s'eliminin.

La persistència s'ha verificat aturant els contenidors amb **docker-compose down** i tornant-los a iniciar amb **docker-compose up**, comprovant posteriorment que les dades continuaven disponibles.

4. Configuration

La configuració del sistema es gestiona mitjançant variables d'entorn per evitar l'ús de valors hardcodejats dins del codi i del fitxer `docker-compose.yml`.

Els valors es defineixen en un fitxer `.env`, on es troben paràmetres com el port de l'aplicació (`APP_PORT`) i les credencials de la base de dades (`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`).

El fitxer `docker-compose.yml` utilitza aquestes variables mitjançant expressions com `${APP_PORT}` o `${POSTGRES_USER}`, que Docker Compose substitueix automàticament pels valors definits al `.env`.

En el servei `backend`, aquestes variables s'utilitzen per configurar l'aplicació Python, indicant el port on ha d'escoltar i com s'ha de connectar a la base de dades (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`).

En el servei `db`, les variables defineixen la configuració inicial de PostgreSQL, com l'usuari, la contrasenya i el nom de la base de dades.

5. Health Checks, depends_on i Restart Policies

5.1 Health Checks

Els health checks permeten a Docker saber si un servei està realment funcionant correctament, no només si el contenidor ha arrancat. Sense health checks, Docker considera un servei com a "en funcionament" en el moment que el procés arrenca, però això no garanteix que estigui llest per rebre peticions.

Cada servei utilitza una comprovació adequada al seu tipus:

- **Nginx**: comprova que respon peticions HTTP correctament
- **Backend**: usa la ruta `/health` de l'aplicació Python
- **PostgreSQL**: utilitza `pg_isready`, una eina pròpia de PostgreSQL

5.2 depends_on amb condition: service_healthy

El `depends_on` amb `condition: service_healthy` garanteix l'ordre correcte d'arrencada dels serveis. Sense aquesta configuració, Nginx podria intentar connectar-se al backend abans que aquest estigués llest, provocant errors d'inici. Gràcies als health checks, cada servei espera que el servei del qual depèn estigui completament operatiu.

L'ordre d'arrencada resultant és:

db (sa) → backend (sa) → nginx

5.3 Restart Policies

La política `restart: always` assegura que si un contenidor falla inesperadament, Docker el reinicia automàticament. Això és fonamental en entorns de producció on es necessita alta disponibilitat, evitant que un error puntual deixi el servei inaccessible fins que algú ho detecti manualment.

Week 10

1. Recursos de Kubernetes

Deployment Un Deployment defineix com s'ha d'executar una aplicació dins del clúster: quina imatge usar, quantes rèpliques tenir i com actualitzar-la. És necessari perquè gestiona automàticament els pods, assegurant que sempre hi hagi el nombre de rèpliques desitjat. Si un pod falla, el Deployment en crea un de nou automàticament sense intervenció manual.

Service Un Service exposa un conjunt de pods perquè siguin accessibles, ja sigui des de dins del clúster o des de l'exterior. És necessari perquè els pods tenen IPs que canvien cada vegada que es reinicien. El Service proporciona una IP i un nom fixos que no canvien mai, independentment de quants pods hi hagi al darrere.

ConfigMap Un ConfigMap emmagatzema configuració en forma de clau-valor, separant la configuració del codi. És necessari per evitar hardcodejar valors com ports, noms de base de dades o usuaris dins dels Deployments. Això permet canviar la configuració sense haver de reconstruir les imatges.

2. Com es comuniquen els pods?

Els pods es comuniquen entre ells mitjançant els Services, usant el **nom del servei** com a adreça. Kubernetes té un DNS intern que resol automàticament aquests noms a la IP correcta.

Per exemple, quan el backend vol connectar-se a la base de dades, simplement usa `db:5432`. Kubernetes resol `db` a la IP del Service de PostgreSQL, independentment de quants pods hi hagi o quina IP tinguin.

backend → db:5432 → Service "db" → Pod PostgreSQL

3. Com accedeixen els clients externs?

Depenent del tipus de Service, l'accés exterior funciona diferent:

Tipus	Ús	Exemple al projecte
Cluster IP	Només accessible des de dins del clúster	backend, db
NodePort	Accessible des de fora mitjançant un port fixe.	nginx pel port 30080

En el nostre cas, l'únic punt d'entrada exterior és Nginx mitjançant el port **30080**. El backend i la base de dades només són accessibles des de dins del clúster, seguint el principi de mínim exposició.

4. Comportament de l'escalat

L'escalat a Kubernetes es fa modificant el nombre de rèpliques d'un Deployment:

```
kubectl scale deployment nginx --replicas=3
```

Quan s'escala cap amunt, Kubernetes crea nous pods automàticament fins arribar al nombre desitjat. Quan s'escala cap avall, elimina els pods sobrants de forma ordenada. En tot moment el Service reparteix el tràfic entre totes les rèpliques disponibles.

A més, Kubernetes implementa **self-healing**: si un pod falla o s'elimina, el Deployment detecta que no té el nombre de rèpliques desitjat i en crea un de nou automàticament, sense cap intervenció manual.

5. Add Resource Limits and Probes

S'han configurat **resource requests** i **resource limits** als deployments per controlar l'ús de CPU i memòria de cada contenidor.

- Els **requests** defineixen els recursos mínims que Kubernetes reserva per a cada pod en el moment de la planificació.
- Els **limits** estableixen el màxim de recursos que un contenidor pot consumir en execució.

Aquesta configuració és essencial en entorns de producció, ja que evita que un contenidor monopolitzi els recursos del clúster i degradi el rendiment de la resta de serveis.

S'han afegit **readinessProbe** i **livenessProbe** als serveis principals per millorar la resiliència del sistema.

- La **readinessProbe** verifica si el contenidor està preparat per rebre tràfic. En cas de fallada, Kubernetes deixa d'enviar-li peticions fins que torni a estar disponible, sense eliminar el pod.
- La **livenessProbe** comprova si l'aplicació continua en funcionament. Si falla de forma consecutiva, Kubernetes reinicia el contenidor automàticament.

Pel que fa a la implementació, el backend i Nginx utilitzen **sondes HTTP**, mentre que PostgreSQL utilitza **pg_isready**, ja que la base de dades no exposa cap endpoint HTTP.

Week 11

1. Terraform

S'ha escollit **Terraform** com a eina d'Infrastructure as Code perquè segueix un enfocament **declaratiu**. Això vol dir que definim l'estat final que volem tenir al clúster Kubernetes, com els deployments, services, configmaps i volums, i Terraform s'encarrega de crear o actualitzar els recursos necessaris.

En canvi, **Ansible** segueix un enfocament més **procedural**, on s'especifiquen passos concrets a executar en ordre. Per aquest projecte, Terraform és més adequat perquè la infraestructura de Kubernetes es pot descriure clarament com a codi i es pot reproduir fàcilment amb `terraform apply`.

A més, Terraform permet utilitzar variables, outputs i un fitxer d'estat, facilitant el control dels canvis i fent que la infraestructura sigui més fàcil de mantenir i versionar al repositori Git.

2. Desplegament de la infraestructura

Per desplegar la infraestructura des del codi, primer cal tenir instal·lades les eines necessàries, com per exemple docker, kubectl, terraform i Minikube.

Un cop instal·lades les eines, s'actualitza el repositori fent un git pull. Després s'arrenca el clúster local amb la comanda `minikube start`, que permet crear i executar un clúster Kubernetes local dins la màquina virtual, simulant un entorn real de Kubernetes. A continuació s'entra a la carpeta de Terraform, `cd terraform`, s'inicialitza Terraform amb `terraform init`, aquest comandament descarrega els proveïdors necessaris, com el provider de Kubernetes, revisem els canvis que es crearan, `terraform plan`, i finalment es desplega la infraestructura, `terraform apply`

Opcionalment, es pot verificar el funcionament amb la següent comanda per veure el nombre de pods creats i el seu estat. → `kubectl get pods`

3. Local deployment workflow

El flux de desplegament està dividit en dues parts: **CI a GitHub Actions** i **CD local a Minikube**.

Quan es fa un `git push` al repositori, GitHub Actions executa automàticament el pipeline de CI. Aquest pipeline valida els fitxers Terraform i construeix les imatges Docker del backend i de Nginx. Després, aquestes imatges es pugen a Docker Hub amb el tag corresponent.

GitHub Actions no executa `terraform apply`, ja que els runners de GitHub no tenen accés al clúster Minikube local. Per aquest motiu, el desplegament final es fa manualment des de la màquina local.

D'aquesta manera, la part de CI garanteix que el codi, les imatges i la infraestructura són vàlids, mentre que la part de CD local desplega els recursos al clúster Minikube.

4. Image tags, variables and outputs

Les imatges Docker generades pel pipeline de CI es versionen mitjançant tags (`v1`, `v2`, `v3`, etc.). Quan GitHub Actions construeix una nova imatge, aquesta es publica automàticament a Docker Hub amb el tag definit al workflow de CI.

El tag utilitzat durant el desplegament es configura al fitxer `variables.tf`, mitjançant les variables:

- `backend_image`
- `nginx_image`

Aquestes variables permeten modificar fàcilment la versió de les imatges desplegades sense necessitat de canviar la resta de fitxers Terraform.

A més, `variables.tf` també conté altres configuracions reutilitzables, com:

- ports
- rèpliques
- configuració de PostgreSQL
- NodePort de Nginx

Terraform també utilitza `outputs.tf` per mostrar informació útil després del desplegament, com:

- URL del backend
- servei de la base de dades
- accés extern a Nginx

Això facilita la verificació de la infraestructura un cop desplegada.

5. CI/CD pipeline

El pipeline de CI/CD s'executa automàticament quan es fa un **push** a la branca principal del repositori.

Durant aquest procés, GitHub Actions:

- valida la configuració Terraform
- construeix les imatges Docker del backend i de Nginx
- publica les noves imatges a Docker Hub

Les imatges es versionen mitjançant tags (**v1**, **v2**, **v3**, etc.). Per actualitzar una versió, cal modificar manualment el tag definit al fitxer **ci.yml** i posteriorment actualitzar el mateix tag al fitxer **variables.tf** utilitzat per Terraform.

Un cop el pipeline finalitza correctament, la nova versió es pot desplegar localment al clúster Minikube executant **terraform apply**.

Aquest procés permet mantenir separades la fase de validació i construcció automàtica (CI) del desplegament local al clúster Kubernetes (CD).

6. Múltiples entorns (dev i staging)

S'han creat dos fitxers **.tfvars** per gestionar entorns separats amb el mateix codi Terraform.

dev.tfvars defineix un entorn de desenvolupament amb menys rèpliques i recursos, pensant en estalviar recursos a la màquina local. **staging.tfvars** simula un entorn més proper a producció, amb més rèpliques per verificar el comportament sota càrrega.

Per desplegar cada entorn s'especifica el fitxer de variables corresponent:

```
terraform apply -var-file="dev.tfvars"
terraform apply -var-file="staging.tfvars"
```

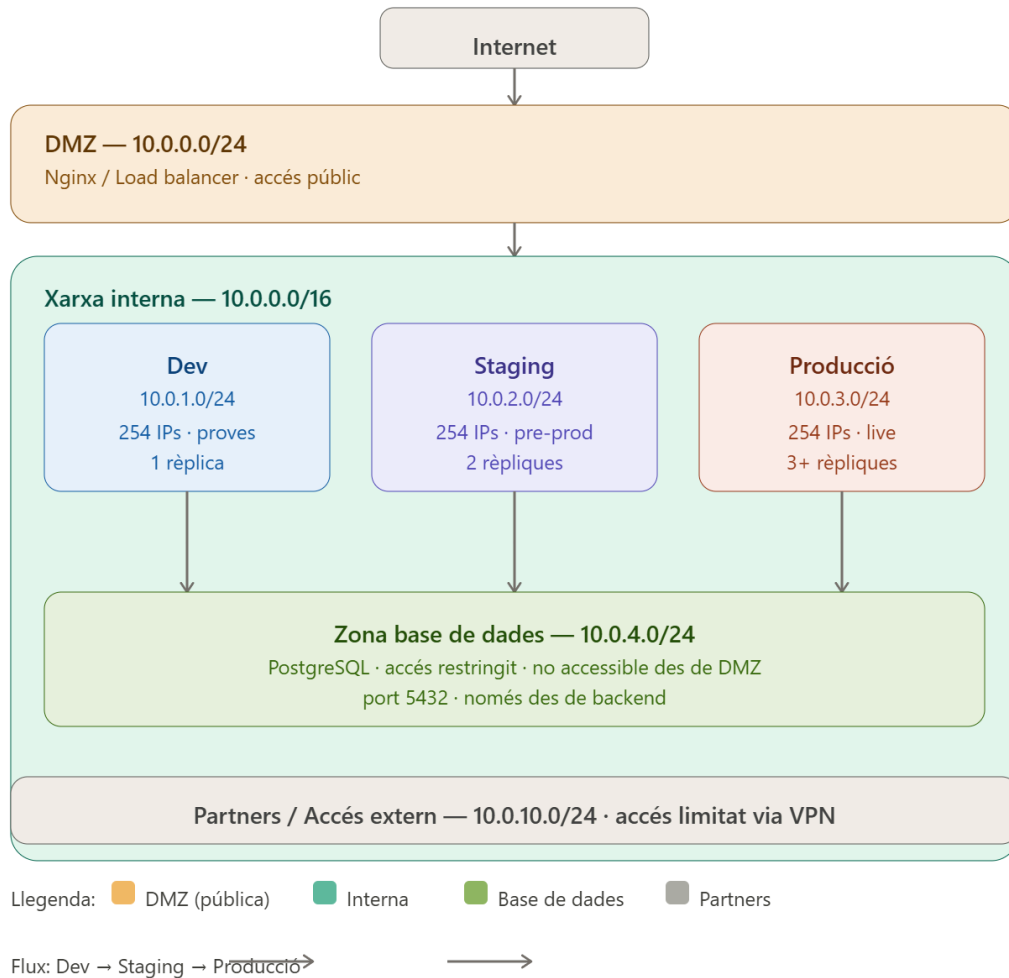
Si no s'especifica cap fitxer, Terraform usa els valors **default** definits al **variables.tf**.

Com s'assegura que staging es prova abans de producció?

El flux correcte és: els canvis es desenvolupen i proven primer a **dev**. Un cop validats, es desplacen a **staging**, que simula l'entorn de producció. Només si staging funciona correctament es promouien els canvis a producció. Això garanteix que cap canvi arriba a producció sense haver estat provat en un entorn equivalent.

Week 12

1. Diagrama de Xarxa



2. Pla d'adreces IP (CIDR)

L'organització utilitza el rang **10.0.0.0/16** com a espai d'adreces principal. Això proporciona fins a 65.534 adreces IP, suficients per al creixement futur de GreenDevCorp.

Aquest rang es subdivideix en subxarxes **/24**, cadascuna amb fins a 254 adreces:

Subxarxa	CIDR	Ús	IPs disponibles
DMZ	10.0.0.0/24	Nginx/ Load Balancer	254
Desenvolupament	10.0.0.1/24	Entorn dev	254
Staging	10.0.0.2/24	Entorn preproducció	254
Producció	10.0.0.3/24	Entorn live	254
Base de dades	10.0.0.4/24	PostgreSQL	254
Partners	10.0.0.10/24	Accés extern limitat	254

Per què /24 per a cada subxarxa?

254 adreces és més que suficient per a cada entorn en una empresa de 20 persones com GreenDevCorp. Usar subxarxes més petites complicaria innecessàriament la gestió, i usar-ne de més grans desperdiciaria adreces.

Per què 10.0.0.0/16 i no un rang més petit?

Reservar un /16 permet afegir nous entorns o serveis en el futur sense haver de redissenyar tota l'arquitectura de xarxa. És una decisió de previsió de creixement.

3. Fronteres de seguretat

Tràfic permès:

Origen	Destí	Port	Motiu
Internet	Nginx (DMZ)	80	Accés públic web
Nginx	Backend	8000	Proxy invers
Backend	PostgreSQL	5432	Consultes a la BD
Partners	Nginx (DMZ)	80	Accés limitat via VPN

Tràfic bloquejat:

Origen	Destí	Motiu
Internet	Backend	El backend no és accessible directament
Internet	PostgreSQL	La BD mai és accessible des de fora
Nginx	PostgreSQL	Nginx no necessita accedir a la BD
Partners	Backend/DB	Accés limitat únicament al frontend

Com es prevenen les males configuracions?

S'aplica el principi de **"default deny"**: tota comunicació està bloquejada per defecte mitjançant la NetworkPolicy `deny-all`. Qualsevol nou servei que s'afegeixi al clúster no tindrà accés a res fins que s'afegeixi explícitament una NetworkPolicy que ho permeti. Això evita que errors de configuració obrin accessos no desitjats accidentalment.

4. Serveis base

DNS (Domain Name System)

DNS tradueix noms de domini llegibles per humans (com `google.com`) a adreces IP que les màquines entenen. Sense DNS, caldria memoritzar la IP de cada servei.

En una organització permet que els serveis interns tinguin noms fixes independentment de quina IP tinguin. Si un servidor canvia d'IP, només cal actualitzar el DNS i tot continua funcionant. En el nostre projecte, Kubernetes té el seu propi DNS intern que permet als contenidors trobar-se pel nom del servei, com `db` o `backend`.

DHCP (Dynamic Host Configuration Protocol)

DHCP assigna automàticament adreces IP als dispositius quan es connecten a la xarxa. Sense DHCP, un administrador hauria de configurar manualment la IP de cada dispositiu.

En una organització és imprescindible a partir d'un cert nombre d'empleats. Quan un treballador connecta el seu portàtil, DHCP li assigna automàticament IP, màscara de xarxa, porta d'enllaç i DNS, sense cap intervenció manual.

NTP (Network Time Protocol)

NTP sincronitza els rellotges de tots els dispositius d'una xarxa. Sense NTP, cada servidor tindria el seu propi temps que s'aniria desviant gradualment.

El temps sincronitzat és fonamental per a la seguretat, ja que els certificats SSL/TLS tenen dates de caducitat que es verifiquen comparant temps entre dispositius. També és essencial per diagnosticar problemes consultant logs de diversos servidors alhora.

5. Management de la identitat

Autenticació vs Autorització

Autenticació és el procés de verificar **qui ets**. Per exemple, introduir un usuari i contrasenya per iniciar sessió. El sistema confirma que ets qui dius ser.

Autorització és el procés de verificar **què pots fer**. Un cop autenticat, el sistema comprova quins recursos o accions tens permesos. Per exemple, un desenvolupador pot accedir als servidors de dev però no als de producció.

En resum: l'autenticació obre la porta, l'autorització decideix a quines habitacions pots entrar.

Identitat centralitzada

LDAP (Lightweight Directory Access Protocol) és un protocol per emmagatzemar i consultar informació d'usuaris de forma centralitzada. Funciona com una base de dades d'usuaris, grups i permisos que altres serveis poden consultar per autenticar.

Active Directory és la implementació de Microsoft de LDAP. És el sistema més utilitzat en empreses per gestionar usuaris, ordinadors i permisos de forma centralitzada.

SSO (Single Sign-On) permet als usuaris autenticar-se una sola vegada i accedir a múltiples serveis sense tornar a introduir credencials. Millora tant la seguretat com l'experiència d'usuari.

La identitat centralitzada soluciona el problema de gestionar credencials separades per a cada servei. Sense ella, cada aplicació té els seus propis usuaris i contrasenyes, cosa que dificulta la gestió i augmenta el risc de seguretat.

Estratègia d'identitat per a GreenDevCorp

Per a una empresa de 20 persones com GreenDevCorp, la recomanació seria implementar un sistema de **SSO basat en LDAP** com OpenLDAP o un servei al núvol com Google Workspace o Microsoft Azure AD.

Per què?

- Gestió centralitzada de tots els usuaris en un sol lloc
- Quan un empleat marxa, es desactiva un sol compte i perd accés a tot
- Els desenvolupadors s'autentiquen una sola vegada per accedir a tots els serveis

Trade-offs:

- Afegeix complexitat inicial de configuració
- Si el servidor d'identitat cau, cap usuari pot autenticar-se. Cal alta disponibilitat.

Una empresa petita podria sobreviure sense identitat centralitzada, però a partir de 10-15 persones la gestió manual de credencials es torna inviable i insegura.

Week 13

1. Prova d'integració completa

Aquest challenge consisteix a verificar que tota la infraestructura es pot desplegar des de zero a partir del codi IaC (Terraform), sense cap intervenció manual. És la prova definitiva que la nostra infraestructura és reproducible i automatitzada.

Pas 1 — Neteja completa de l'entorn

S'han eliminat tots els pods i deployments existents per partir d'un estat net:

```
minikube stop
minikube delete
minikube start --driver=docker
kubectl get pods      # Ha de sortir: No resources found in default namespace.
```

Pas 2 — Desplegament des del codi Terraform

Des del directori terraform/ s'han executat les comandes de desplegament:

```
cd ~/green-dev-infra2/terraform
terraform init
terraform plan
terraform apply      # Enter value: yes
```

Pas 3 — Verificació end-to-end

Un cop desplegat, s'ha verificat que tots els serveis estaven en funcionament:

```
kubectl get pods
```

```
gsx@vbox:~/green-dev-infra2/terraform$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-5dcff8cc44-4xbkx            1/1     Running   0           5m
db-795d66476c-b9kvd                 1/1     Running   0           4m59s
nginx-76899589f5-8pf2m              1/1     Running   0           5m1s
```

```
kubectl get networkpolicies      # Han de sortir les 3 polítiques
```

```
gsx@vbox:~/green-dev-infra2/terraform$ kubectl get networkpolicies
NAME                                POD-SELECTOR  AGE
allow-backend-to-db                 app=db        6m15s
allow-nginx-to-backend              app=backend   6m15s
deny-all                           <none>        6m16s
```

Les polítiques de xarxa estan definides i aplicades. En un entorn real amb un CNI com Calico o Cilium s'enforçarien automàticament. Minikube amb el driver Docker té limitacions de compatibilitat amb CNI avançats, però les polítiques estan correctament definides i desplegades

El temps de desplegament, des de terraform apply fins Running ~5 min

Pas 4 — Verificació de la comunicació interna

Les dades persisteixen entre redesplegaments de Terraform mentre el clúster Minikube estigui actiu. En cas de `minikube delete`, cal re-inicialitzar la base de dades manualment amb el següents passos:

```
kubectl exec -it <nom-pod-db> -- psql -U user -d testdb
```

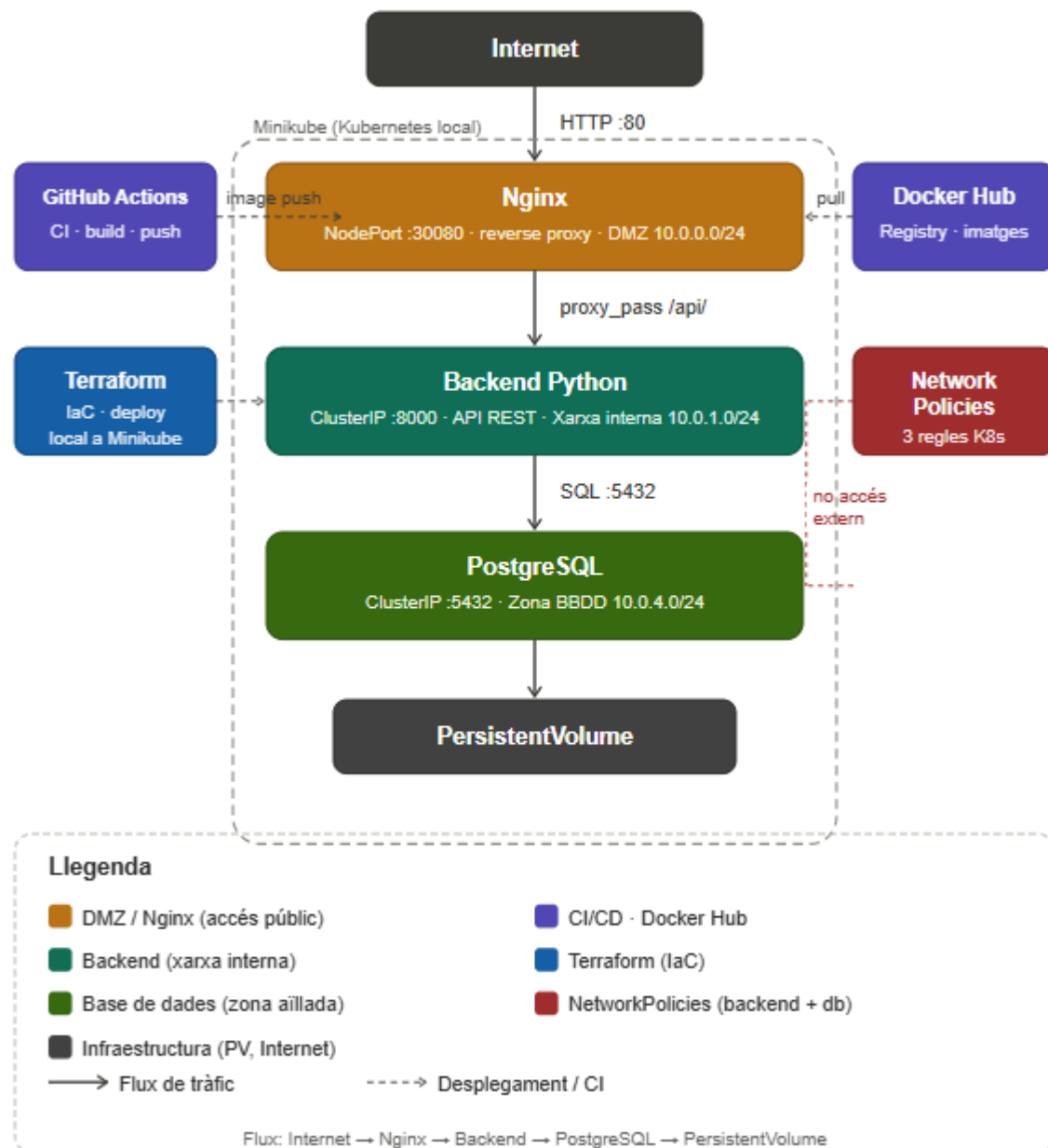
```
# Crea la taula i afegeix dades
```

```
CREATE TABLE IF NOT EXISTS students ( id SERIAL PRIMARY KEY, name  
VARCHAR(100) NOT NULL ); INSERT INTO students (name) VALUES ('Nicolas'), ('Marçal');  
\q
```

En un entorn de producció real s'automatitzaria amb un init container o un Kubernetes Job que executi les migracions en arrencar.

```
gsx@vbox:~$ curl http://192.168.49.2:30080/api/  
CI/CD updated version  
1 - Nicolas  
2 - Marçalgsx@vbox:~$
```

2. Diagrama d'Arquitectura



Tot el sistema corre dins d'un clúster Minikube (Kubernetes local). El tràfic extern entra per l'NGINX, que actua de reverse proxy cap al backend. El backend accedeix a la base de dades PostgreSQL, que és inaccessible des de l'exterior gràcies a les NetworkPolicies.

2.1 Documentació dels Components

Servei	Funció	Imatge Docker	Port
nginx	Reverse proxy i servidor web estàtic. Punt d'entrada per als clients externs.	marcalfigueras/nginx-gsx:v2	80 (NodePort)
backend	API REST en Python. Gestiona la lògica de negoci i accedeix a la BD.	marcalfigueras/simple-app-gsx:v2	8000 (ClusterIP)
db	Base de dades PostgreSQL. Emmagatzema les dades dels estudiants.	postgres:15-alpine	5432 (ClusterIP)

Nginx

- Serveix el fitxer index.html com a pàgina principal.
- Actua de reverse proxy: les peticions a /api/ es reenvien al backend (http://backend:8000).
- Exposat externament via NodePort per permetre l'accés des de fora del clúster.
- Imatge basada en nginx:latest, lleugera i optimitzada per a contingut estàtic.

Backend (Python)

- Servidor HTTP en Python 3.11-alpine (imatge mínima per reduir la superfície d'atac).
- Exposa els endpoints: /health (estat del servei), /students (llistat d'estudiants).
- Configurat amb variables d'entorn per a la connexió a la base de dades.
- Executa com a usuari no-root (appuser) per seguir el principi de mínim privilegi.

Base de dades (PostgreSQL)

- PostgreSQL 15 Alpine, lleugera i robusta.
- Configurat amb variables d'entorn (POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB).
- Dades persistents gràcies a un PersistentVolumeClaim (PVC) — els reinicis de pod no perden les dades.
- Accessible únicament des del backend gràcies a les NetworkPolicies.

2.2 Operational runbook

Com desplegar una nova versió

```
# 1. Fer canvis al codi i fer push
git add . && git commit -m 'feat: nova funcionalitat'
git push origin main

# 2. La CI (GitHub Actions) construeix i puja la nova imatge
# Esperar que el workflow passi (veure a GitHub Actions)

# 3. Actualitzar el tag manualment amb kubectl
kubectl set image deployment/backend \
backend=marcalfigueras/simple-app-gsx:<nou-tag>
```

Com escalar un servei

```
# Escalar el nginx a 3 rèpliques
kubectl scale deployment nginx --replicas=3

# Verificar l'escalat
kubectl get pods --watch

# Tornar a 1 rèplica
kubectl scale deployment nginx --replicas=1
```

Com revisar els logs

```
# Logs d'un pod específic
kubectl logs <nom-del-pod>

# Logs en temps real (seguiment)
kubectl logs -f <nom-del-pod>

# Logs d'un deployment (agrupa tots els pods)
kubectl logs deployment/backend
```

Com destruir i netejar l'entorn

```
cd ~/green-dev-infra2/terraform
terraform destroy # Elimina tots els recursos de Kubernetes

# O eliminar el clúster sencer
minikube delete
```

2.3 Guia de Troubleshooting

Problema: Un pod no arrenca

```
# 1. Veure l'estat dels pods
kubectl get pods

# 2. Veure els events i motiu de l'error
kubectl describe pod <nom-del-pod>

# 3. Veure els logs del pod
kubectl logs <nom-del-pod>
kubectl logs <nom-del-pod> --previous # Si ja ha reiniciat
```

Causes freqüents: imatge incorrecta, variables d'entorn que falten, problemes de connexió a la BD.

Problema: El servei X no pot arribar al servei Y

```
# 1. Verificar que el servei existeix
kubectl get services

# 2. Entrar al pod i fer curl manualment
kubectl exec -it <pod-origen> -- sh
curl http://<nom-servei>:<port>/health

# 3. Verificar les NetworkPolicies
kubectl get networkpolicies
kubectl describe networkpolicy <nom>
```

Causes freqüents: nom del servei incorrecte, NetworkPolicy massa restrictiva, port incorrecte.

Problema: ImagePullBackOff

```
kubectl describe pod <nom-del-pod>
# Buscar la línia 'Failed to pull image'
```

Causes freqüents: la imatge no existeix al registre, credencials de Docker Hub incorrectes, tag inexistent.

Problema: Terraform apply falla

```
# Verificar que minikube està actiu
minikube status

# Revisar l'estat de Terraform
terraform show
terraform plan # Veure quins canvis proposa
```


3. Reflexió Individual

Nicolas:

Un dels aspectes més difícils va ser la configuració de Kubernetes, especialment la part dels fitxers YAML, ja que moltes vegades feies `apply` i els serveis no arrencaven correctament, o bé fallaven les proves de liveness i readiness. A més, depurar aquests errors era bastant complicat.

El que més m'ha sorprès de la infraestructura moderna, a part de Kubernetes, ha estat la part de CI/CD. En altres assignatures ens havien explicat el concepte, però mai l'havíem vist "en acció". Normalment, quan fem projectes en grup, no acostumem a fer molts commits petits, sinó un únic commit gran, i això sovint provoca conflictes o errors en fer el merge. Amb CI/CD aquest procés és molt més ràpid, segur i controlat.

Una part que hauria fet diferent és dedicar més temps des del principi a entendre millor la teoria i l'arquitectura de les tecnologies utilitzades. Tot i que entenia els conceptes generals del que estàvem fent, hi havia moments en què em perdia o no tenia del tot clar per què estàvem utilitzant una solució concreta i no una altra.

També hauria intentat fer més proves petites durant el desenvolupament, en comptes d'avançar moltes coses de cop. Per exemple, en la part de Kubernetes hauria estat millor comprovar primer només el Deployment del backend, després afegir el Service, després la base de dades i finalment Nginx. Això hauria facilitat molt la detecció d'errors, ja que quan tot falla alhora és més difícil saber si el problema ve de la imatge, del port, de les variables d'entorn, del Service o de les proves.

Aquesta pràctica també m'ha ajudat a entendre millor la diferència entre simplement executar contenidors i gestionar una infraestructura real. Amb Docker Compose aixequés diversos contenidors junts, però amb Kubernetes ja treballes amb conceptes com estat desitjat, rèpliques, resiliència, serveis interns i actualitzacions controlades. Això m'ha fet veure que una aplicació no només ha d'estar programada, sinó que també ha d'estar preparada per desplegar-se, recuperar-se de fallades i escalar si augmenta el tràfic. També he après que la documentació i l'organització són gairebé tan importants com la configuració tècnica. Si els fitxers Terraform, els manifests de Kubernetes o els workflows de CI/CD no estan ben explicats, és molt difícil que una altra persona pugui reproduir l'entorn. Per això ara entenc millor la importància de tenir infraestructura versionada, instruccions clares i commits més petits.

Tot i això, la càrrega de treball d'altres assignatures feia difícil poder aprofundir tant com m'hauria agradat en aquestes tecnologies.

Encara així, considero que són eines molt importants de cara al futur, ja que actualment moltes empreses utilitzen sistemes cloud, contenidors i automatització per desplegar i mantenir aplicacions de forma eficient i escalable.

També he entès millor per què les empreses inverteixen tant en infraestructura i automatització. Aquestes eines permeten evitar errors humans i escalar aplicacions de forma consistent i reproduïble.

En el futur m'agradaria aprofundir més tant en automatització com en plataformes cloud com AWS, ja que crec que són tecnologies molt importants actualment i aporten moltes facilitats de gestió, escalabilitat i control. També m'interessaria veure com es gestiona una aplicació molt més gran, amb múltiples serveis, així com aprendre a configurar una VPC amb diferents subxarxes (subnets) per controlar millor el tràfic i els accessos a Internet.

Marçal:

Quan va començar aquesta pràctica, la paraula "contenedor" em semblava abstracta i llunyana. Sis setmanes més tard, puc dir que he passat d'instal·lar Docker per primera vegada a entendre per què empreses com Google o Netflix gestionen la seva infraestructura d'aquesta manera.

El moment que més em va costar va ser entendre la relació entre totes les capes de la infraestructura. Docker és relativament intuïtiu, però quan vas afegint Docker Compose, Kubernetes i Terraform, cada capa afegeix la seva pròpia complexitat. El que em costava no era cada eina per separat, sinó entendre com encaixaven totes juntes. Per exemple, em va costar veure per què necessitàvem Terraform si ja teníem els fitxers YAML de Kubernetes. La resposta, que Terraform permet gestionar tot des d'un sol lloc de forma declarativa i versionada, va trigar a fer-se evident.

El que més m'ha sorprès és fins a quin punt la filosofia "tot és codi" impregna la infraestructura moderna. Abans pensava que configurar servidors era quelcom manual i difícil de repetir. Descobrir que amb una sola comanda terraform apply pots recrear tota una infraestructura des de zero, amb contenidors, xarxes, polítiques de seguretat i configuració inclosa, canvia completament la perspectiva. La reproductibilitat no és un luxe, és un requisit.

Dedicaria molt més temps a la fase de disseny abans d'implementar. En diverses ocasions vam haver de tornar enrere perquè una decisió presa a la setmana 8 condicionava el que podíem fer a la setmana 10. Per exemple, no haver pensat des del principi en la comunicació entre serveis va fer que afegir el proxy invers al Nginx fos més complicat del necessari. En un projecte real, el disseny previ estalvia moltes hores de feina.

Abans associava DevOps simplement amb "fer deploys més ràpid". Ara entenc que és una cultura que elimina la frontera entre qui escriu el codi i qui el posa en producció. Cada tecnologia que hem après resol un problema concret d'aquesta frontera: Docker garanteix que el codi funciona igual a tot arreu, Kubernetes garanteix que es recupera sol si falla, Terraform garanteix que la infraestructura és reproducible, i GitHub Actions garanteix que cada canvi es valida automàticament. Juntes, fan que un equip petit pugui gestionar una infraestructura que abans requeria un departament sencer.

M'agradaria aprofundir en la gestió de secrets, ja que al llarg de la pràctica hem hagut de posar contrasenyes a fitxers de configuració, cosa que no és acceptable en producció. Eines com HashiCorp Vault permeten gestionar secrets de forma segura i centralitzada. També m'interessa Helm, que simplifica molt el desplegament d'aplicacions complexes a Kubernetes, i el món del cloud públic, especialment com es tradueixen tots aquests conceptes a plataformes com AWS o Google Cloud.